QAOA for Portfolio Optimization

Quantum Optimization • Qiskit Fall Fest x IIIT Delhi • Track 2

A reproducible six-stock, cardinality-constrained portfolio experiment

Core result: the observed best QAOA depth was p=2 (approximation ratio 0.980765), while λ=1.0 gave the strongest penalty-sweep balance: 97.1% valid samples and ratio 0.9791.

Prepared with Python 3.12.15 and Qiskit 2.5.2 • Local simulator experiments

1. Problem and objective

We select exactly K = 3 assets from n = 6 synthetic stocks. Each binary variable x■ indicates whether stock i is selected.

Objective: minimize C(x) = q·x■Σx – μ■x + λ(Σx■ – K)², with q = 0.5.

• μ is generated uniformly between 0.05 and 0.25.

• Σ = AA■, where A has normally distributed entries with scale 0.1; this produces a positive-semidefinite covariance-like matrix.

• Random seed = 2026, so the generated instance is reproducible.

• The penalty term discourages bitstrings that do not select exactly three stocks.

QAOA maps the binary objective to a cost Hamiltonian using x■ = (1 – Z■)/2, then alternates cost and mixer operations.

2. Classical benchmark and Hamiltonian validation

• Enumerated all 2■ = 64 bitstrings and identified 20 feasible portfolios with exactly three selected stocks.

• The best feasible portfolio is bitstring 011001, corresponding to zero-based stock indices (1, 2, 5).

• Best feasible objective value: –0.50301469.

• At λ = 1, the enumerated minimum and maximum objective values were approximately –0.50301469 and 9.0.

Hamiltonian cross-check: all 64 bitstrings were tested against the direct classical objective.

Validation check	Observed result
Number of bitstrings checked	64
Maximum absolute energy difference	2.998 × 10■¹■
Status	All 64 checks passed

The near-zero discrepancy is consistent with floating-point numerical precision and supports the correctness of the Pauli expansion.

3. QAOA workflow

• Constructed the cost Hamiltonian as a SparsePauliOp.

• Ran QAOA using a local statevector simulator and COBYLA classical optimization.

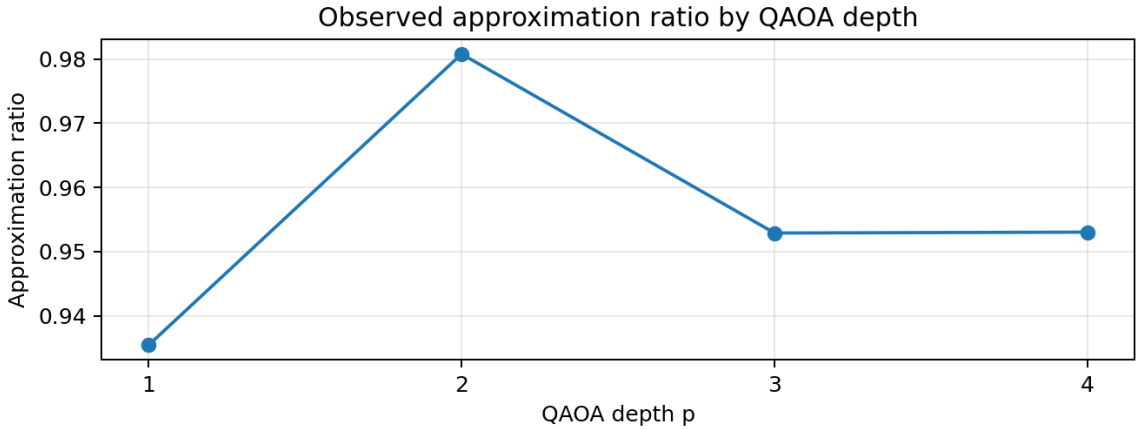
• Sampled 4,096 bitstrings for the p=1 experiment.

• Compared measured objective values with the exact classical enumeration.

• Swept QAOA depth p = 1, 2, 3, 4 using three restarts per depth and a maximum of 180 optimizer iterations per restart.

The approximation ratio used was r = (Cmax – ■C■) / (Cmax – Cmin), with Cmin and Cmax calculated over all 64 bitstrings for the same penalty setting. Higher values are better under this definition.

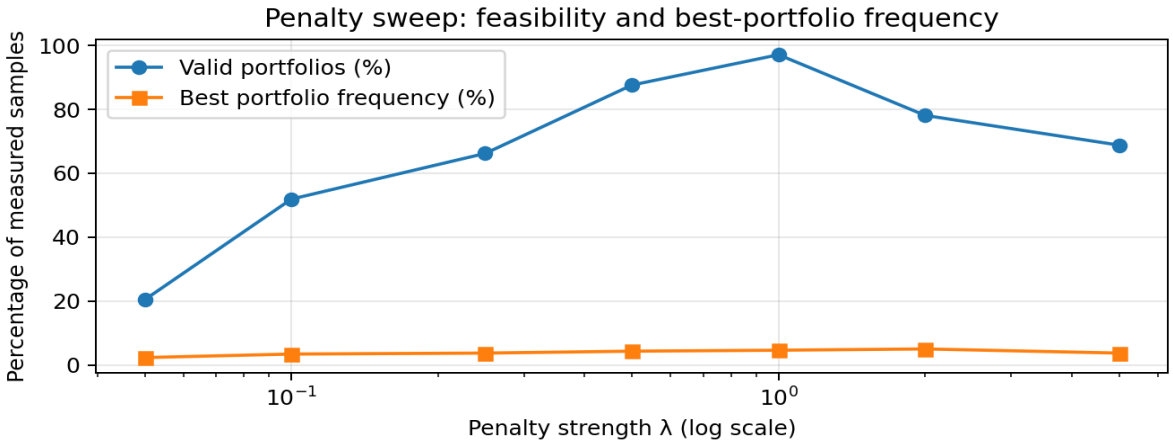
4. Results by QAOA depth



Depth p	Mean objective ■C■	Approximation ratio	Optimizer evaluations*
1	0.110714	0.935418	38
2	–0.320226	0.980765	180
3	–0.055274	0.952884	180
4	–0.056589	0.953023	180

*The p=2, 3, and 4 runs reached the 180-evaluation limit in the reported run. Results are observed outcomes, not proof of global convergence.

5. Penalty-strength sweep



λ	Valid fraction	Best 011001 frequency	Ratio
0.05	20.4%	2.3%	0.8884
0.10	51.9%	3.4%	0.9096
0.25	66.2%	3.7%	0.9278
0.50	87.6%	4.3%	0.9618
1.00	97.1%	4.6%	0.9791
2.00	78.1%	5.0%	0.9534
5.00	68.8%	3.7%	0.9566

λ = 1.0 gave the highest observed validity fraction and approximation ratio. λ = 2.0 had the highest frequency for the exact best bitstring, showing that these metrics measure different things.

6. Interpretation, limitations, and next steps

• **Penalty trade-off:** too little penalty permits infeasible portfolios; very large penalties can distort the optimization landscape and reduce observed feasibility.

• **Depth:** p=2 achieved the highest observed ratio, but higher-depth runs hit the iteration cap and may benefit from more optimization.

• **Best-solution frequency:** the exact best portfolio appeared in 4.6% of samples at λ=1.0 and 5.0% at λ=2.0; a high average ratio does not mean the best bitstring dominates samples.

• **Limitations:** small synthetic instance (six assets), simulator-only results, and finite optimizer restarts/iterations. Findings should not be generalized to real-world portfolios.

• **Next steps:** increase optimizer budget, compare more initializations, and report feasibility and objective quality together.

7. Reproducibility and project repository

Repository: github.com/SnehilK3372/qiskit-fall-fest-2026-IIITD

Notebook sequence

• 01_classical_benchmark.ipynb — seeded data, brute-force enumeration, feasible portfolios.

• 02_portfolio_hamiltonian.ipynb — Pauli expansion and all-64-bitstring validation.

• 03_portfolio_qaoa.ipynb — p=1 simulation and classical comparison.

• 04_depth_experiments.ipynb — p=1–4 comparison.

• 05_penalty_experiments.ipynb — λ sweep, feasibility, and best-portfolio frequency.

Environment reported: Python 3.12.15, Qiskit 2.5.2. Experiments use local simulation; an IBM Quantum API key is not required for these runs.

Conclusion: on this seeded toy instance, p=2 and λ=1.0 gave the strongest observed approximation-ratio results in their respective experiments, with λ=1.0 also producing the highest measured feasible-sample fraction.

Before submission, verify that the GitHub repository contains the final notebooks, README, and this PDF, and rerun all notebooks top-to-bottom in the intended environment.